

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

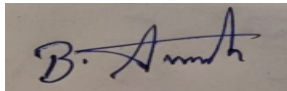
Assessment Tool Weightage: 40%

Code of Conduct:

I Avinash B (192511193) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in dark ink, appearing to read "B. Avinash", is written on a light-colored rectangular background.

Question 2: Automated Video Surveillance with Alert System

Constraints: (A) detect motion/unusual activity, (B) real-time alerts, (C) standard CCTV feeds, (D) minimize false positives in dynamic environments (swaying trees, lighting flicker).

End-to-end pipeline

1. Input acquisition: `cv2.VideoCapture` reads the standard CCTV stream (RTSP/file/USB), satisfying constraint C with no special hardware.
2. Preprocessing: convert to grayscale and apply a light Gaussian blur to suppress sensor noise before background modeling.
3. Motion detection: use adaptive background subtraction (`cv2.createBackgroundSubtractorMOG2` or `KNN`) rather than simple frame differencing — it models each pixel with a Gaussian mixture that adapts to gradual, repetitive change (swaying trees, slow lighting drift), automatically absorbing it into the 'background' over time.
4. Mask cleanup: apply morphological opening/closing (`cv2.morphologyEx`) to remove small noise blobs from the foreground mask and fill small gaps within real moving objects.
5. Contour filtering: `cv2.findContours` on the cleaned mask, discarding any contour below a minimum area threshold — the main defense against tiny, irrelevant movement (leaves, insects, noise).
6. Temporal validation: require a candidate motion event to persist across several consecutive frames before it counts as 'real' — this filters out single-frame flicker that a one-shot check would otherwise flag.
7. Alerting: once a validated event passes both the area and persistence checks, log the timestamp, save a snapshot/clip, and push a real-time alert.
8. Visualization: draw bounding boxes around active motion regions for the live operator view.

Justification

MOG2 background subtraction plus morphological cleanup, area filtering, and temporal persistence together target every named false-positive source (repetitive background motion, sensor noise, brief flicker) rather than relying on any single filter. All stages are lightweight per-frame operations, so the pipeline comfortably sustains real-time processing on standard CCTV frame rates and hardware, directly satisfying the real-time alerting requirement alongside the false-positive constraint.

Question 6: Real-Time Object Detection on Mobile Device

Constraints: (A) detect objects via the camera, (B) real-time performance, (C) limited CPU and memory.

End-to-end pipeline

1. Image acquisition: capture frames from the device camera via `cv2.VideoCapture(0)` (or the platform camera API bridged into OpenCV).
2. Preprocessing: downscale each frame to a small fixed resolution (e.g., 320×240) before detection — detector runtime scales directly with pixel count, so this is the single cheapest way to cut cost.
3. Lightweight detection model: run a MobileNet-SSD (or similarly mobile-optimized) model through OpenCV's DNN module (`cv2.dnn.readNet`) instead of a heavy detector like full YOLO or Faster R-CNN — MobileNet's depthwise-separable convolutions cut parameters and FLOPs substantially versus standard CNN architectures.
4. Inference optimization: use quantized (INT8) model weights to shrink memory footprint and speed up CPU inference, and enable any available on-device acceleration backend (e.g., NNAPI).

5. Frame-rate management: decouple camera capture from inference using a producer-consumer threaded design, and skip inference on some frames (process every 2nd/3rd frame) if needed, so the displayed camera feed stays smooth even when detection can't keep up with the native frame rate.
6. Post-processing: apply non-max suppression and a confidence threshold to raw detection outputs.
7. Display: overlay bounding boxes and labels on the live camera feed.

Justification

Each optimization targets a different stage of cost: input downscaling reduces per-pixel work, the MobileNet-SSD architecture reduces per-parameter compute, and quantization reduces memory footprint — together directly answering the CPU/memory constraint. Threaded capture/inference and optional frame skipping directly answer the real-time constraint by ensuring inference latency never blocks the camera feed itself, so the app always feels responsive even if the effective detection rate is lower than the camera's native FPS.

Question 9: Document Scanner Application

Constraints: (A) capture document images, (B) enhance readability, (C) output clean scanned images.

End-to-end pipeline

1. Image acquisition: capture a still image of the document via the mobile camera.
2. Noise reduction: apply a bilateral filter rather than a plain Gaussian blur — bilateral filtering smooths noise while preserving sharp text edges, whereas a naive blur would also blur the text itself.
3. Document boundary detection: convert to grayscale, run Canny edge detection, find contours (`cv2.findContours`), and identify the largest quadrilateral via `cv2.approxPolyDP` — this locates the document's actual edges within the photo, separating it from the background surface.
4. Perspective correction: compute a 4-point perspective transform (`cv2.getPerspectiveTransform` + `cv2.warpPerspective`) from the detected corners to produce a flat, rectangular top-down view, correcting for the camera not being held perfectly perpendicular to the page.
5. Illumination correction: apply adaptive thresholding (`cv2.adaptiveThreshold`) instead of a single global threshold — this compensates for uneven lighting across the page (e.g., a shadow on one side), producing consistent black-text-on-white-background output.
6. Sharpening/contrast enhancement: optional unsharp masking or CLAHE for a final clarity boost.
7. Output: save the flattened, high-contrast result as the final 'scanned' image/PDF.

Justification

Perspective correction directly solves the core problem that a handheld photo (unlike a flatbed scan) is rarely taken perfectly square-on to the page. Adaptive thresholding specifically handles the uneven, directional lighting typical of handheld capture far better than a single global threshold would, and bilateral filtering denoises without sacrificing the text sharpness that OCR/readability ultimately depends on — together these stages turn a raw, skewed, unevenly-lit photo into a clean, readable scanned page.

Question 13: Multi-Face Tracking System

Constraints: (A) detect multiple faces, (B) track them across frames, (C) maintain identity.

End-to-end pipeline

1. Image acquisition: `cv2.VideoCapture` for live video input.
2. Face detection: run a fast detector (Haar cascade for lightweight scenarios, or OpenCV's DNN-based res10 SSD face detector for better robustness) to get bounding boxes for every visible face in a detection frame.
3. Track initialization: for each newly detected face not already matched to an existing tracker, start a dedicated per-object tracker (e.g., `cv2.TrackerCSRT_create()` or `cv2.TrackerKCF_create()`) and assign it a persistent numeric ID.
4. Frame-to-frame tracking: on frames between full detection passes, update each active tracker to get its new predicted position — far cheaper than re-running full-frame detection every single frame, since trackers exploit small frame-to-frame motion instead of re-searching the whole image.
5. Periodic re-detection and matching: every N frames, re-run full face detection and match new detections to existing tracked IDs using bounding-box overlap (IoU) or simple appearance similarity — this corrects tracker drift, adds trackers for faces newly entering the frame, and removes trackers for faces that have left.
6. Identity continuity across occlusion: if a tracked face is briefly lost (e.g., occluded) and a similar face reappears nearby shortly after, use appearance-similarity matching to reassign the same ID rather than creating a new one.
7. Output: draw a bounding box and ID label for every actively tracked face each frame.

Justification

Combining cheap per-object trackers with periodic full re-detection balances speed against robustness: running full multi-face detection on every frame would be far more expensive as the number of simultaneously tracked faces grows, while relying on tracking alone (with no periodic re-detection) would let drift accumulate unchecked and would never notice new faces entering the scene. The periodic re-matching step is also what actually delivers constraint C — maintaining a stable identity per face rather than just a stable bounding box.

Question 15: Real-Time Edge Detection System

Constraints: (A) capture video, (B) detect edges in real time, (C) display results efficiently.

End-to-end pipeline

1. Image acquisition: `cv2.VideoCapture(0)` or a video file/stream source.
2. Preprocessing: convert each frame to grayscale (edge detection doesn't need color, and working on one channel instead of three cuts computation roughly threefold), then apply a small Gaussian blur to suppress high-frequency sensor noise that would otherwise produce spurious edges.
3. Edge detection: apply Canny edge detection (`cv2.Canny`) with tuned low/high hysteresis thresholds. Canny is chosen over simpler operators (plain Sobel/Laplacian) because its non-maximum suppression and hysteresis thresholding produce thin, well-connected, low-noise edge maps at only a modest additional computational cost.
4. Throughput optimization: downscale the frame before edge detection if the native camera resolution exceeds what's needed for display/analysis, and run capture and edge detection on separate threads (producer-consumer) so a momentarily slower processing pass doesn't stall the camera read.
5. Display: show the edge map directly via `cv2.imshow`, optionally blended back onto the original color frame for a combined view.

Justification

Grayscale conversion and blurring are both cheap and directly improve Canny's output quality by removing noise-driven false edges before detection even runs. Canny specifically is worth its modest extra cost over simpler gradient operators because it delivers a materially cleaner, thinner edge map for essentially the same order of computation — which matters here because the goal is efficient *and* clean display, not just a fast but noisy result. Downscaling and threaded capture/processing are what actually guarantee the real-time frame rate holds as resolution or system load increases.